RDARuntime: An OS for AI Accelerators

Benjamin Glick
SambaNova Systems, Inc
Palo Alto, California, USA
benjamin.glick@sambanova.ai

Arjun Sabnis
SambaNova Systems, Inc
Palo Alto, California, USA
arjun.sabnis.@sambanova.ai

Renate Kempf
SambaNova Systems, Inc
Palo Alto, California, USA
renate.kempf.@sambanova.ai

Arnav Goel
SambaNova Systems, Inc
Palo Alto, California, USA
arnav.goel.@sambanova.ai

Aarti Lalwani
SambaNova Systems, Inc
Palo Alto, California, USA
aarti.lalwani.@sambanova.ai

Guoyao Feng
SambaNova Systems, Inc
Palo Alto, California, USA
guoyao.feng.@sambanova.ai

Kiran Ranganath
SambaNova Systems, Inc
Palo Alto, California, USA
kiran.ranganath.@sambanova.ai

ABSTRACT

Today's supercomputers are more heterogeneous than ever before. As the share of AI workloads in data centers continues to grow, the share of GPUs and AI-specific hardware grows with it. AI accelerators are different from traditional hardware, affecting all aspects of system design, from data-center scale to single-chip scale. AI accelerators are much more efficient than CPUs or GPUs for some HPC workloads, especially in AI for Science. They also add complexity to system architecture, management, and programming. Although runtime frameworks are critical to reducing system complexity, there is little literature describing AI accelerator runtimes. In this paper, we introduce RDARuntime - an AI-specific OS tailored for the development and operation of SambaNova's reconfigurable dataflow architecture. We discuss the architecture, our design decisions, and some of the results we have achieved, along with some lessons we have learned while helping to deploy the Reconfigurable Dataflow Unit (RDU) to production environments.

CCS CONCEPTS

• **Software and its engineering** → **Designing software**; **Operating systems**; • **Hardware** → *Functional verification*; Networking hardware; • **Computing methodologies** → **Distributed artificial intelligence**; • **Computer systems organization** → *Systolic arrays*; **Dependable and fault-tolerant systems and networks**.

KEYWORDS

AI accelerators, operating systems, runtime frameworks, distributed learning, system management

ACM Reference Format:
Benjamin Glick, Arjun Sabnis, Renate Kempf, Arnav Goel, Aarti Lalwani, Guoyao Feng, and Kiran Ranganath. 2023. RDARuntime: An OS for AI Accelerators. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023), November 12–17, 2023, Denver, CO, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3624062.3624235

SC-W 2023, November 12–17, 2023, Denver, CO, USA

ACM ISBN 979-8-4007-0785-8/23/11…$15.00
https://doi.org/10.1145/3624062.3624235

1 INTRODUCTION

Today's computing environments are full of AI/ML applications. In response to the growing share of AI, data centers have become more heterogeneous. 37% (185/500) of the latest TOP500 supercomputers use GPUs or other accelerators [4], and commercial data centers have grown their use of accelerators as well [26]. AI-specific accelerators can be more efficient at running AI/ML model workloads than traditional hardware. AI accelerators often have custom instruction sets and don't have the same software ecosystem as CPUs or even GPUs. In this paper, we present our work on RDARuntime, an operating system for the SambaNova Reconfigurable

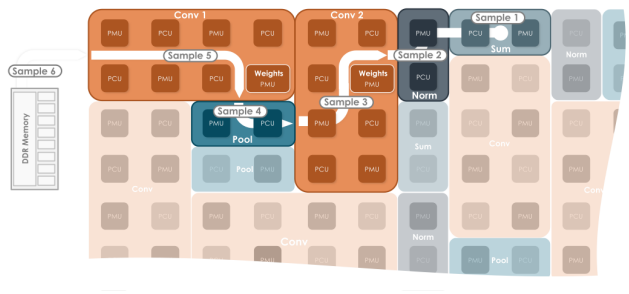

Figure 1: ML workload running on a section of an RDU

Dataflow Unit (RDU). RDARuntime is not an operating system in the traditional sense, but it solves the typical problems that an OS would solve in the context of the RDU.

An RDU is an AI accelerator based on a Coarse-Grained Reconfigurable Architecture (CGRA). Conceptually, a CGRA is between a GPU, which has a number of fixed cores that perform a limited set of instructions, and an FPGA, which can be configured at the logic-gate level. A CGRA is made up of multi-purpose logic units. [30] The RDU's architecture, called the Reconfigurable Dataflow Architecture (RDA), is made up of three primary components: the Pattern Compute Unit (PCU), which can perform arithmetic operations, the Pattern Memory Unit (PMU), which is a memory bank, and the Address Generator and Coalescing Unit (AGCU), which enables off-chip communication. The components within an RDU are connected by an on-chip network. All RDUs within a system can communicate over a proprietary protocol [1]. RDUs support spatial multitasking, where sections of the RDU are used for different tasks. This spatial approach to computation (as opposed to the temporal approach a GPU might take) scales well and is well suited for AI workloads. Two versions of the RDU are currently in production: the SN10 and SN30. Both versions are available in a fixed configuration with 8 RDUs and an x86-based host in an integrated system, which is called DataScale. Each SN30 has 640MB of on-chip memory, over 1TB of off-chip memory, and 688 TFLOPS in the bf16 format [3] [23].

The RDA has advantages over CPU or GPU hardware for certain AI/ML workloads [12], but RDUs are not natively programmable or accessible from any existing OS. To solve this problem, we have built the RDARuntime framework.

RDARuntime has the same goals as any other OS: to abstract the hardware, offer easy interaction with the system, and manage the hardware. RDARuntime is used internally for hardware development, and is part of SambaNova's products. That means the users of RDARuntime have different needs. For hardware development, direct access to hardware is paramount, while for production software, providing useful layers of abstraction is important. The high-level goals of RDARuntime are:

- Manage the hardware and ensure system stability
- Provide simple APIs for complex hardware accesses in a multi-tenant environment
- Minimize the OS's performance overhead

Other goals include ease of development, scalability, support for different families of RDUs, and support for both training and inference on the same system, at the same time.

2 RELATED WORK

While AI accelerators are a recent development, the RDARuntime stack has learned from other OSes and accelerator runtimes. In particular, UNIX design philosophies and some aspects of GPU runtimes have been influential.

2.1 GPUs

The NVIDIA driver and CUDA Runtime (CUDART) framework is an example of another accelerator runtime. At a high level, the CUDART stack's goals are similar to those of the RDARuntime. However, we have taken different approaches to important functionality areas.

2.1.1 Similarities. CUDART introduces "compute capability", which is used to convey GPU compute ability and presence of hardware features, and to identify the GPU in use from the application. [20] Similarly, the RDARuntime kernel advertises RDU hardware features to the application library. This information is particularly useful because it allows us to create a single portable and extensible software stack that works across multiple generations of RDU hardware.

2.1.2 Differences. The NVIDIA driver's hardware management depends on firmware running on a service processor, while RDARuntime handles those tasks in software on the host CPU. A major difference is the way multiple devices are managed. In the GPU framework, each GPU is presented to the host OS as a separate device file. [17] Processes have to explicitly select the specific GPU they would like to use. In the RDARuntime model, a single virtual device is presented to the OS, and processes request a number of abstract compute resources from RDARuntime. This approach allows RDARuntime to efficiently schedule multiple workloads across all the hardware that it manages. RDARuntime doesn't require user code to explicitly list RDUs before choosing which one to use. RDARuntime still offers applications the ability to request specific physical RDU resource assignments, if they want, through tile affinity and RDU I/O channel hints, which are analogous to CPU affinity. [19]

In userspace, there are also some major differences between CUDART and RDARuntime. CUDART stops at lower-level APIs, [17] while RDARuntime extends higher, to APIs needed by ML frameworks. Additionally, the CUDA runtime's app execution requires additional communication between the CUDA frontend in userspace and the driver framework in kernelspace to execute applications, while RDARuntime maps the necessary regions of the RDU's address space directly into applications' virtual memory. This approach gives applications secure, direct access to the RDU.

2.2 OS-Bypass Software

Latency is critical for hardware access through an OS. Access through the kernel is a performance bottleneck which has been well explored through networking software. One way to eliminate this bottleneck is to bypass the kernel and allow userspace programs to access hardware. The OS-bypass approach is particularly developed in data-plane networking. [10] Another interesting approach is to relocate the kernel's duties to domain-specific hardware, removing the kernel from the performance path. [25]

AI accelerators are not network equipment, but they are similarly latency-sensitive. Especially in the case of inference workloads, fast configuration of the accelerator is paramount. We have learned from the DPDK (Data Plane Development Kit) project [2], whose approach is to split accesses into two phases. First is a relatively time-insensitive configuration step where DPDK maps the hardware it's configuring into a program using DMA mappings. [7] The second step grants userspace processes ownership of those sections of the hardware. The more latency-optimized userspace software can then directly access the hardware without any interaction from the kernel. This allows for low-latency, real-time configuration of hardware in the data path. [6]

RDARuntime takes a similar approach. For initialization, device management, and part of the interrupt handling framework, we use a kernel driver. This persistent privileged entity can abstract and manage hardware. The driver implements system calls with particular attention to latency. We preconfigure the device for application execution at system boot, and then at runtime, we dynamically map requested sections of the hardware to the requesting program's virtual memory. This is a hybrid between a kernel-module-based device driver architecture and an OS-bypass approach. It affords us the system integrity of kernel-based resource management, while providing the low-latency runtime configuration that we need for inference workloads.

2.3 Design Principles of Modern OSes

RDARuntime learns from CPU operating systems in two main areas: memory management and scheduling. GNU/Linux has influenced our design in those areas. The RDARuntime memory allocator uses an algorithm similar to many implementations of 'malloc', for a low likelihood of requiring remapping or reallocation. [16]

The RDARuntime kernel maintains a topology-aware virtual-to-physical RDU mapping. Each process is allocated a contiguous set of virtual RDUs, which can map to non-contiguous or non-unique physical RDUs. For compute resource scheduling, RDARuntime manages a resource/process schedule using the virtual-to-physical mapping. In some cases, an RDU is being used by only one process, while for some advanced use cases, preemption, oversubscription, and other schedule management is required. The algorithm that Linux uses to schedule CPU processes is more complex than the RDARuntime algorithm, primarily because CPUs have more well-developed hardware support for context-switching than RDUs. We use a simplified method which covers just the RDU stack's needs.

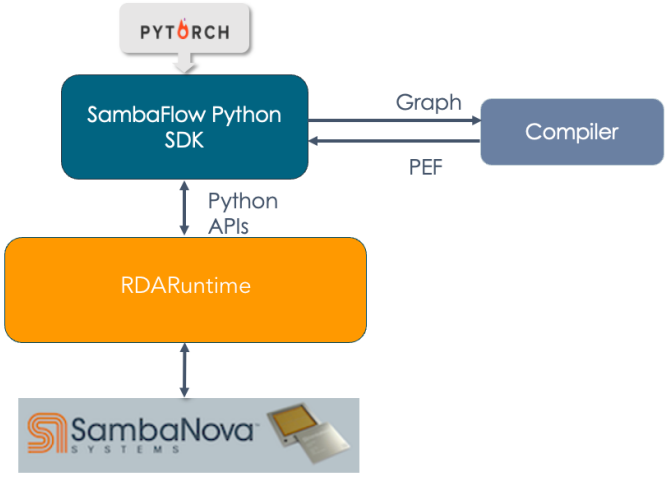

Figure 2: RDARuntime stack is between ML frameworks, RDA compiler, and SambaNova RDU hardware

3 RDARUNTIME ARCHITECTURE

RDARuntime serves as an intermediary between the SambaFlow compiler stack, the ML framework, and the RDU. The compiler stack interacts with RDARuntime through an executable file called a PEF, which describes the parts of the model that run on the RDU. The CPU parts of the model are passed to RDARuntime by an ML framework like PyTorch. RDARuntime supports functions both upwards, providing services to users that facilitate application runs, and downwards, to manage the hardware and ensure system integrity. RDARuntime also helps RDUs interact with 3rd party hardware, such as storage or network resources.

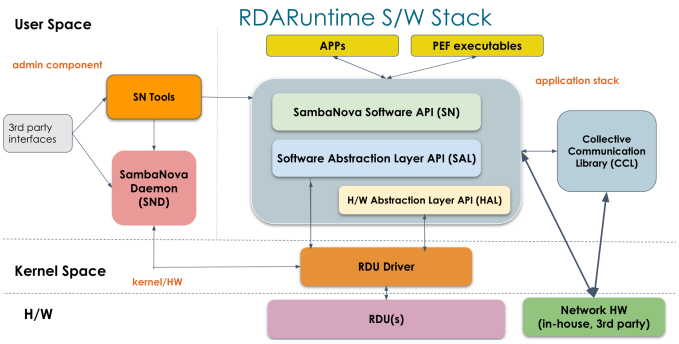

Figure 3: RDARuntime stack main components: SND on left, core application stack and CCL on right, kernel across the bottom

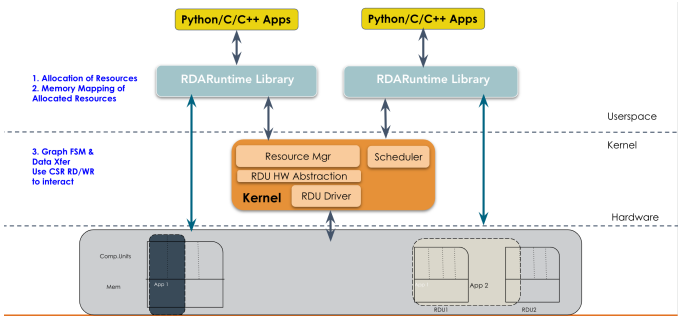

Figure 4: Multi-tenancy on RDARuntime

RDARuntime has three primary components: the kernel, the SambaNova Daemon (SND), and the application stack. The kernel offers low-level APIs that extend the host's operating system with access to the RDU and performs privileged tasks, like managing multi-tenancy. SND is a user-level program which handles system initialization and fault management. SND includes a framework for secure interaction with RDARuntime, called the SambaNova Management Layer (SNML). The application stack is a system library of APIs that allow users to run ML models. Part of the application stack is a Collective Communication Library (CCL) which orchestrates data-parallel scale-out.

3.1 Runtime Kernel: Resource Allocation, Scheduling, and Hardware Abstraction

A primary goal of RDARuntime is simple and secure multi-tenancy. Several users may want to use the same RDU hardware at the same time, and RDARuntime mediates requests. The only part of RDARuntime that has a trusted view of which resources belong to which processes is the kernel. The Kernel Resource Manager (KRM) tracks the hardware resources and validates resource requests. The kernel component presents all RDU resources and services through a single device file. This is in contrast to most GPU drivers, where a device file is created for each GPU [17]. RDARuntime operates from a single virtual device for two main reasons: resources can seamlessly come and go upon hardware events or user requests, and resource scheduling becomes simpler.

In what we call "appliance mode", an application opens the RDARuntime application library and requests RDU hardware. The userspace library forwards those requests to the kernel, which attempts to allocate and schedule the job. If allocation is unsuccessful, the kernel returns a descriptive error message. If allocation succeeds, a return code is passed to the allocating process, and RDARuntime requests that the kernel memory map the resources into the application's virtual memory. The kernel memory maps only the resources that it has allocated to the userspace process, resulting in two main advantages: first, the process has access only to resources that the KRM has allocated to it, and second, the memory-mapped approach gives the userspace library direct access to the hardware, which ensures low-latency [21].

Appliance mode gives sufficient user-to-user isolation for most purposes. RDARuntime supports another means of providing isolation environments, with a slight increase of performance overhead. We call it "cloud mode". In cloud mode, many of the features of the userspace RDARuntime library are moved to the kernel component, as a secure command server instead of library calls. Userspace processes are not given direct access to any hardware and instead submit command requests to the kernel's command server, which analyzes the requests and executes the commands that are deemed not to be malicious [8]. Cloud mode is especially useful for zero-trust situations, for example where different tenants of a single system come from different organizations.

In addition to handling multi-tenancy and isolation, the kernel component performs a number of other functions. Some of them are required by the hardware's design, while others provide software-only features. They include:

- **PCIe and character device drivers.** The RDU has a PCIe interface that we use for configuration and host/device communication, so we must register a PCIe device driver with the RDU to use features like PCIe config space.
- **Interrupt handling and event generation.** The kernel event manager uses the kernel, which has a global view of all RDU resources
- The **Hardware Abstraction Library (HAL)** allows higher levels of the stack to use a simple set of hardware-agnostic verbs (including `read` and `write`) to interact with the RDU. The HAL allows us to audit when an RDU is accessed and provides a building block that the rest of the RDARuntime stack uses to access RDUs.

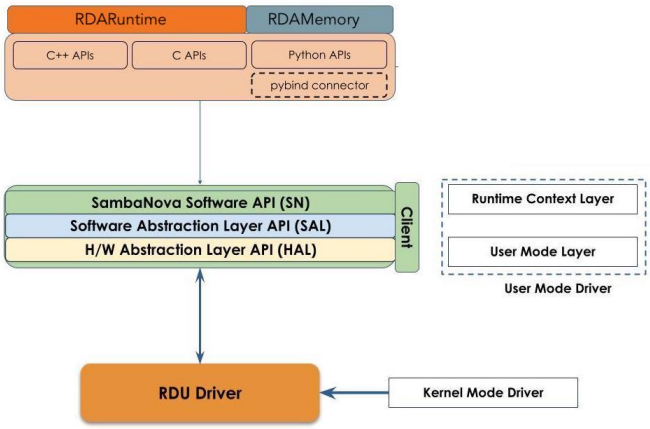

Figure 5: RDARuntime application stack architecture: SambaRuntime frontend, SN Client layer, and kernel-mode driver

3.2 Application Library Component

The RDARuntime application stack is responsible for performantly orchestrating applications, which may be ensembles of small to medium sized models (high-throughput) or single large models (high-performance).

3.2.1 Single RDU Applications: Application Stack Basics. The application stack consists of three layers - the frontend, the user mode driver, and the kernel mode driver.

The **RDARuntime frontend** is a multi-lingual suite of libraries that provides compute and memory interfaces to a dynamic pool of RDUs. It also includes observability and debuggability interfaces.

The **user mode driver** acts as an interface between the user-facing frontend and the hardware-facing kernel mode driver. It contains the SambaNova (SN) Layer, the Software Abstraction Layer (SAL) and the Hardware Abstraction Layer (HAL). A Client API layer connects components of the user-mode driver and external components.

The **SN layer**:

- Maps compile-time (PEF) and link-time (API) **resource requirements** into virtualized compute, memory, and I/O structures used by the SAL control plane.
- Maps compile-time and link-time **application metadata** into compute and data transfer configuration structures used by the SAL data-plane.

The **SAL** is a software abstraction of hardware control-plane and data-plane features, including:

- A **resource manager** for virtual to physical mappings of hardware commands and resources
- An **event manager** for asynchronous delivery of hardware interrupts
- A **memory manager** of host and device memory pools
- A **graph control service** that manages RDU compute resources
- A **data transfer service** that manages RDU I/O channels for communication acceleration

The **HAL** programs hardware components to take actions on RDUs. The APIs in the HAL abstract platform-level differences such as register layouts.

The **kernel mode driver** manages the RDU devices and peripherals in the I/O subsystem. It has knowledge of the resources allocated to a given application. This driver allocates RDUs and device memory, forwards hardware interrupts, and configures I/O channels between combinations of accelerators and the host.

At application runtime:

(1) A user creates a session for a PEF through the frontend.
(2) The SN layer reads compile-time and link-time resource metadata, and passes virtual resource requirements to the SAL.
(3) SAL requests hardware resources from the kernel driver.
(4) The kernel driver allocates resources and prepares the hardware for program execution.
(5) A user launches the application from the frontend.
(6) The SAL uses compile-time hints and link-time heuristics to launch the application [13].
(7) The SN layer orchestrates the sections in the application, based on the application's instruction schedule and compile/link-time metadata.
(8) The SAL executes the application on the RDU and orchestrates data transfer in parallel. Interactions with the hardware are handled by the HAL.

3.2.2 RDUs as Disaggregated Resources. The RDU system is only available in a fixed configuration. The smallest system "building block" is 8 RDUs connected to an x86 host. It is intended to be used exclusively for AI workloads. It is easiest to achieve high utilization when running an AI application that runs entirely on RDU systems. However, there are also several use cases for mixed workloads where a traditional scientific computing program is informed by an ML model. We call these ML models that help inform a traditional HPC job surrogate models [5]. Surrogate models can reduce the total amount of computation needed for a useful scientific result by guiding the simulation based on a mix of the data it produces and data known from other sources, for example empirical studies. [15]

In these scenarios, it is necessary to treat the RDU as a disaggregated resource, and to dispatch work to it via a high-speed network when the HPC part of the workload needs to interact with the ML model. Each part of the workload can use the compute resources that are most efficient for its demand, while also allowing for high utilization of both RDU and CPU/GPU resources. This approach has been particularly successful in Cognitive Simulation (CogSim) [31].

3.2.3 Parallelism for Very Large Models. The RDARuntime stack supports two flavors of scale-out: model parallel and data parallel. Model parallel can be used for inference or training jobs, while data parallel is training-specific.

Data parallelism in AI/ML is thematically similar to data parallelism in HPC. In a data-parallel program, the same instruction is applied to multiple different pieces of data at the same time. Then a reduction process, which allows the program to make use of all the data generated or processed in parallel, is applied to the whole program. [28] In a data-parallel machine-learning model, we run several copies of the same program through the forward and backward sections of the model, and then we perform an all-reduce operation which averages the gradients in place, resulting in identical gradients on each replica. Finally, the application proceeds to the optimizer step and continues to its next epoch. This approach works only for training applications. [24] It has been shown that this data-parallel approach provides a reasonable scaling factor for ML training applications. [32] [11] [18]. We use data parallelism for training if an entire model can be efficiently mapped into one RDU.

Model parallelism, on the other hand, is more similar to task parallelism in HPC. In task parallelism, different hardware compute units perform different tasks in parallel. This approach grants more flexibility than data parallelism because it is not limited to parts of a program that perform the exact same compute operations on different data, but can also be applied to sections of a program where different operations are required on either the same or different data. The same logic applies to model parallelism in ML models. [9] Through model-parallelism, we can scale up our hardware resource usage in a wider range of situations than through data parallelism, including both inference and training applications where we want processes to communicate outside of the gradient sync process. This flexibility comes at the cost of complexity, because the scope of operations that require cross-processor communication is less limited. In contrast, data parallelism only allows communication for gradient sync. We rely on the Dataflow compiler to efficiently orchestrate model-parallel applications. For large inference applications, we must use model-parallel.

One approach we often take towards parallelism for large training applications at SambaNova is similar to that of the

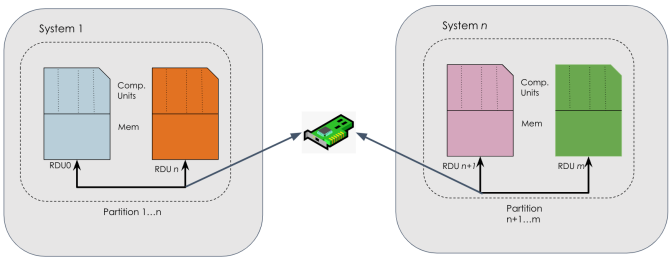

Figure 6: Model-parallel scale-out in RDARuntime

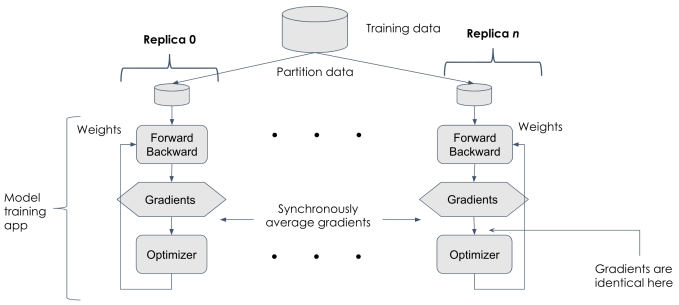

Figure 7: Data-parallel scale-out in RDARuntime

authors of the Megatron model [27]. We combine model parallel and data parallel in one large distributed application. Each RDU within a small group, up to one node (typically 8 accelerators and one x86-based host) runs in a fine-grained group as a single model-parallel application. Then those applications run together in a coarse-grained group acting as a set of replicas for a large, distributed data-parallel application. This approach combines the simplicity of data parallel scaling with the flexibility in terms of model size and hardware resource consumption afforded by model-parallelism. Some results of this approach are described in the **Results** section below. This approach has also been used successfully for non-ML workloads [14].

3.3 Administrative Component: Hardware Management and Access

The SambaNova Daemon (SND) service is the part of RDARuntime that manages the hardware. SND works with APIs provided by the kernel component to access the hardware. SND is a set of C/C++ libraries that run in userspace, as a system service under the `root` user. SND goals are:

- System initialization
- RDU-to-RDU communication fabric management
- Fault management and system hardware diagnostics
- Offering access to the hardware to users

SND supports these functions using a modular architecture. A single SND frontend dispatches requests to various

backend components, each of which has a single specific purpose. Each backend function may also need access to the RDU hardware, which is provided through a unified RDU hardware abstraction library. The system self-diagnostics components each have their own group of functionalities that are responsible only for system diagnostics. For each diagnostic component, the hardware component calls into the self-diagnostic component.

The **RDU device initialization library** manages RDU system boot. It is responsible for bringing RDUs out of reset, initializing their components, checking their health, and getting them ready for applications to use. It also stores system data that other components of SND may need to access, including the RDUs' serial numbers and firmware versions. The library also handles dynamic reconfiguration (DR) to allow for high-availability of RDUs. Some examples of DR operations include responding to RDU hardware exceptions and faults, re-interleaving device memory, re-initializing RDU communication fabrics, and user-requested RDU additions or removals. [22]

The system's **RDU-to-RDU network fabric (RDUConnect)** is also managed by SND on the host. RDUConnect supports a variety of physical networking technologies. The RDUConnect management component of SND initializes the RDU-to-RDU communication fabric, enumerates which routes were successfully initialized, tracks the bandwidth on each route, and performs updates on the fabric.

Both the device management library and the RDUConnect management library have associated diagnostic libraries. These diagnostic libraries run if requested by users at system boot, perform stress tests on the RDU hardware to detect marginal or defective hardware, and report suspected hardware through the SambaNova Fault Management (SNFM) framework.

The **SNFM framework** supports reporting, diagnosing, and analyzing the system error and fault events associated with a DataScale system. It tracks errors in real time, and diagnoses faults when those errors suggest that a hardware problem is likely. SNFM takes automatic recovery actions for certain component failures, and advises corrective action to recover from faults for other failures. Its main goal is to provide a single point of reference for the system's health and manage recoverable faults to allow as much system uptime as possible. It also records a persistent history of faults and errors that can be used for future analysis and diagnosis.

Another component of SND is the **SambaNova Management Layer (SNML)**, which is a platform for managing RDU devices. SNML supports flexible device management of the RDU system. SNML acts as an intermediary for users and tools to interact with the RDUs. It's the primary way that a user may request information or actions from the runtime

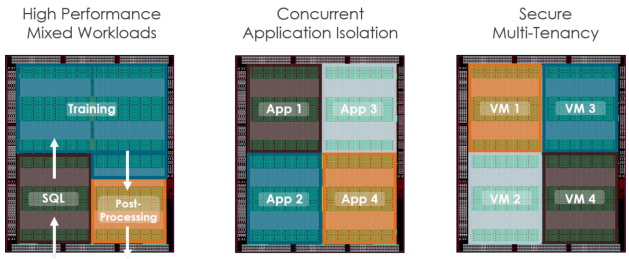

Figure 8: Three approaches towards multi tenancy and virtualization on RDU

stack. Some of the tools it powers include our profiler, configurator, reset tools, and VM provisioning tools. From a user's perspective, SNML is split into three main services.

- A read-only monitoring service, used for gathering information about the configuration and status.
- A read-write server, used for managing RDU system settings. Actions it can take include enabling and disabling certain hardware features, resetting RDUs, and managing SNFM's behavior.
- A service that provides direct access to the hardware. This service is most useful for low-level debugging and functional verification.

Finally, SND orchestrates **RDU virtualization**. RDU virtualization is desirable in a number of situations, including for optimizing usage, elevating isolation, or for high availability (through features like live migration). SND interacts with LibVirt to provide native support for several hypervisors and VM orchestration frameworks. SND facilitates both physical function passthrough virtualization and virtual function passthrough virtualization through Single-Root I/O Virtualization (SR-IOV). This offers similar functionality to GPU virtualization frameworks, with more seamless support for single-device, multi-VM virtualization and LibVirt integration. [29]

4 EMPIRICAL ANALYSIS

Because RDARuntime has varied goals, it's difficult to find a single metric that summarizes the experience of using it. Each component of RDARuntime has different design concerns and compromises, so different metrics are of value to different components. In the application library, latency is the most critical. The kernel component is primarily concerned with security and stability rather than performance, while SND is somewhere in the middle.

We ran applications on one SN30 chip to evaluate latency added by RDARuntime, and we ran multi-RDU model training to characterize scaling.

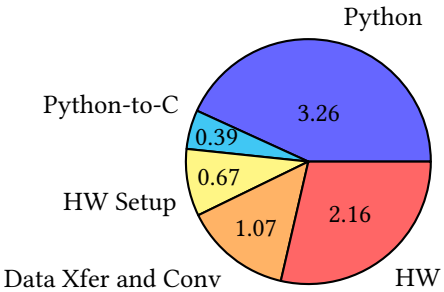

Figure 9: Latency of SambaFlow components (sec over 10,000 iterations)

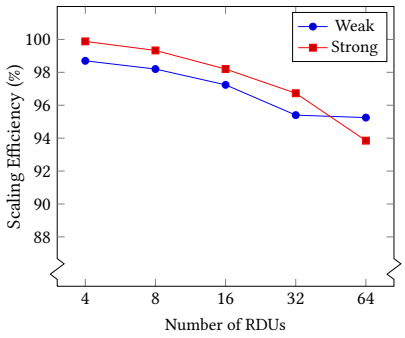

Figure 10: Scaling efficiency on SN30 and RDARuntime

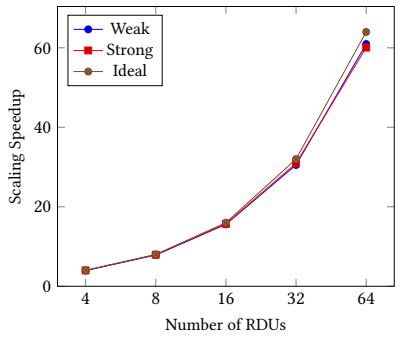

Figure 11: Scaling speeedup on SN30 and RDARuntime

4.1 Single RDU

For the single-chip measurements, we collected latency information with our internal profiling tool while running a logistic regression application for 100 iterations on one SN30 RDU. Figure 9 shows the results.

Each entry in the chart represents time spent, in seconds, over 10,000 iterations. The **Python** section shows time spent in the Python application and PyTorch. The **HW Setup** slice reflects context creation, resource allocation, hardware setup, and cleanup. The **Data Xfer and Conv** slice is time transferring data to the RDU and performing data conversion. **HW** is time spent with applications running on the RDU. The RDU and Python times are highly app-dependent and should be treated as examples. The key observation is that RDARuntime overhead is insignificant when compared the rest of the software stack.

4.2 Multi-RDU

We evaluated scaling with a GPT-13B parameter model across 1-64 SN30 RDUs. The application uses the model-parallel plus data-parallel approach described above. On each SN30 system, two threads are running in a fine-grained model-parallel group. Each model-parallel group serves as a single replica in a coarse-grained data-parallel ensemble. We evaluated strong and weak scaling by varying batch size with number of RDUs. To analyze strong scaling, we used a fixed global batch size of 8192 and varied both the per-worker batch size and the number of RDUs. To analyze weak scaling, we used a fixed per-worker batch size of 128 and varied the global batch size by adding RDUs. The global batch size reflects the total amount of processing that the RDUs need to do, while the per-RDU batch size reflects the amount of processing that each RDU needs to do. Weak scaling efficiency ranged from 98.73% (4 RDUs) to 95.25% (64 RDUs), while strong scaling efficiency ranged from 99.88% (4 RDUs) to 93.85% (64 RDUs). The full data is in Appendix A and summarized in figures 10 and 11. The 1-8 RDU trials ran on one system, and communicated via the RDUConnect fabric. The 1-16 RDU trials ran on one rack, where the 16 RDU trial used Ethernet to communicate across hosts. The 32-64 RDU trials ran across multiple racks, with cross-rack Ethernet switches handling communication. This data was collected in our internal development lab. While we reserved exclusive racks for these trials, the cross-rack switches were not exclusively reserved. Note that scaling depends on model architecture, hyperparameters, and other application-specific factors. These scaling data may not generalize to all applications.

5 FUTURE WORK

In the future, we will investigate in more detail how we can efficiently scale up and scale out both training and inference workloads through data-, tensor-, and model- parallelism. We would like to study utilization and compute efficiency when using RDARuntime for different kinds of workloads, both with the RDU as a disaggregated resource for an HPC workload and with an AI-specific workload. We would also like to perform scaling studies past 64 RDUs and explore how the RDU can directly interface with external I/O, memory, and storage hardware in order to achieve maximum performance and flexibility for varied workloads. Finally, we would like to share more about the challenges raised by the SN40L RDU, which has a more complicated memory and I/O hierarchy.

6 CONCLUSION

We present our experience building RDARuntime, the runtime framework for the RDU. We cover some lessons learned by studying other systems software, and discuss our novel HW/SW architecture, and the design questions that each component raises. We also share a brief analysis of performance metrics such as latency and throughput, and a scaling

study demonstrating the ability to performantly orchestrate data- and model-parallel workloads across many RDUs.

The RDARuntime software stack enables rapid development of several generations of RDU hardware and of novel large language models in excess of 1 trillion parameters. Our architecture efficiently organizes and executes models across hundreds of RDUs with low latency and high throughput. We experimentally demonstrate performance for both scaling-sensitive training and latency-sensitive inference. The design of RDARuntime helps realize these goals by abstracting complexity while offering end-users enough control over important options. These design and implementation decisions help make RDARuntime an effective operating system.

ACKNOWLEDGMENTS

Thank you Fansheng Cheng, Greg Dykema and Blaine Rister for your guidance. Thank you Neal Sanghvi for assisting with figures. Thank you to all RDARuntime contributors, who are listed at https://glick.cloud/rdaruntime-contributors.

REFERENCES

[1] [n. d.]. *Accelerated Computing with a Reconfigurable Dataflow Architecture.* Retrieved July 29, 2023 from https://sambanova.ai/wp-content/uploads/2021/04/SambaNova_Accelerated-Computing-with-a-Reconfigurable-Dataflow-Architecture_Whitepaper_English.pdf

[2] [n. d.]. *Data Plane Development Kit.* Retrieved July 29, 2023 from https://github.com/DPDK/dpdk

[3] [n. d.]. *SambaNova DataScale® SN30.* Retrieved September 14, 2023 from https://sambanova.ai/wp-content/uploads/2022/09/SambaNova_DataSheet_DataScale_SN30_09132022_EN-1.pdf

[4] [n. d.]. *TOP500 HIGHLIGHTS - JUNE 2023.* Retrieved July 29, 2023 from https://www.top500.org/lists/top500/2023/06/highs/

[5] Claudio Angione, Eric Silverman, and Elisabeth Yaneske. 2022. Using machine learning as a surrogate model for agent-based simulations. *Plos one* 17, 2 (2022), e0263150.

[6] Adel Belkhiri, Martin Pepin, Mike Bly, and Michel Dagenais. 2023. Performance analysis of DPDK-based applications through tracing. *J. Parallel and Distrib. Comput.* 173 (2023), 1–19. https://doi.org/10.1016/j.jpdc.2022.10.012

[7] Ivano Cerrato, Mauro Annarumma, and Fulvio Risso. 2014. Supporting Fine-Grained Network Functions through Intel DPDK. In *2014 Third European Workshop on Software Defined Networks.* 1–6. https://doi.org/10.1109/EWSDN.2014.33

[8] Ranen Chatterjee, Ravinder Kumar, Raghunath Shenbagam, Maran Wilson, Conrad Alexander Turlik, Arnav Goel, Arjun Sabnis, and Yannan Chen. 2023. Elevated Isolation of Reconfigurable Data Flow Resources in Cloud Computing. Retrieved July 31, 2023 from https://patentimages.storage.googleapis.com/c3/26/ee/3a19bad1548112/US20230205585A1.pdf Patent No. US20230205585A1, Filed December 12, 2022, Issued June 29, 2023.

[9] Chi-Chung Chen, Chia-Lin Yang, and Hsiang-Yun Cheng. 2018. Efficient and Robust Parallel DNN Training through Model Parallelism on Multi-GPU Platform. *arXiv e-prints* (2018), arXiv–1809.

[10] Ruining Chen and Guoao Sun. 2018. A Survey of Kernel-Bypass Techniques in Network Stack. In *Proceedings of the 2018 2nd International Conference on Computer Science and Artificial Intelligence* (Shenzhen, China) *(CSAI '18).* Association for Computing Machinery, New York, NY, USA, 474–477. https://doi.org/10.1145/3297156.3297242

[11] Murali Emani, Venkatram Vishwanath, Corey Adams, Michael E Papka, Rick Stevens, Laura Florescu, Sumti Jairath, William Liu, Tejas Nama, and Arvind Sujeeth. 2021. Accelerating scientific applications with sambanova reconfigurable dataflow architecture. *Computing in Science & Engineering* 23, 2 (2021), 114–119.

[12] Murali Emani, Zhen Xie, Siddhisanket Raskar, Varuni Sastry, William Arnold, Bruce Wilson, Rajeev Thakur, Venkatram Vishwanath, Zhengchun Liu, Michael E. Papka, Cindy Orozco Bohorquez, Rick Weisner, Karen Li, Yongning Sheng, Yun Du, Jian Zhang, Alexander Tsyplikhin, Gurdaman Khaira, Jeremy Fowers, Ramakrishnan Sivakumar, Victoria Godsoe, Adrian Macias, Chetan Tekur, and Matthew Boyd. 2022. A Comprehensive Evaluation of Novel AI Accelerators for Deep Learning Workloads. In *2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS).* 13–25. https://doi.org/10.1109/PMBS56514.2022.00007

[13] Gregory Frederick Grohoski, Manish K Shah, Raghu Prabhakar, Mark Luttrell, Ravinder Kumar, Kin Hing Leung, Ranen Chatterjee, Sumti Jairath, David Alan Koeplinger, Ram Sivaramakrishnan, et al. 2022. Runtime Patching of Configuration Files. US Patent App. 16/996,666.

[14] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond Data and Model Parallelism for Deep Neural Networks.. In *Proceedings of Machine Learning and Systems*, A. Talwalkar, V. Smith, and M. Zaharia (Eds.), Vol. 1. 1–13. https://proceedings.mlsys.org/paper_files/paper/2019/file/b422680f3db0986ddd7f8f126baaf0fa-Paper.pdf

[15] Peishi Jiang, Nis Meinert, Helga Jordão, Constantin Weisser, Simon Holgate, Alexander Lavin, Björn Lütjens, Dava Newman, Haruko Wainwright, Catherine Walker, and Patrick Barnard. 2021. Digital Twin Earth – Coasts: Developing a fast and physics-informed surrogate model for coastal floods via neural operators. arXiv:2110.07100 [physics.ao-ph]

[16] Poul-Henning Kamp. 1998. Malloc (3) revisited. In *1998 USENIX Annual Technical Conference (USENIX ATC 98).*

[17] David Kirk et al. 2007. NVIDIA CUDA software and GPU parallel computing architecture. In *ISMM*, Vol. 7. 103–104.

[18] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. [n. d.]. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proceedings of the VLDB Endowment* 13, 12 ([n. d.]).

[19] Robert Love. 2003. Kernel korner: CPU affinity. *Linux Journal* 2003, 111 (2003), 8.

[20] Ogier Maitre and Pierre Collet. 2013. *Understanding NVIDIA GPGPU Hardware.* Springer Berlin Heidelberg, Berlin, Heidelberg, 15–34. https://doi.org/10.1007/978-3-642-37959-8_2

[21] Anand Misra, Arnav Goel, Qi Zheng, Raghunath Shenbagam, and Ravinder Kumar. 2021. Time-Multiplexed use of Reconfigurable Hardware. Retrieved July 31, 2023 from https://patentimages.storage.googleapis.com/a0/ac/0c/06792e61002e09/US20220269534A1.pdf Patent No. US20220269534A1, Filed February 25, 2021, Issued August 25, 2022.

[22] Anand Misra, Conrad Alexander Turlik, Maran Wilson, Anand Vayyala, Raghu Shenbagam, Ranen Chatterjee, Pushkar Shridar Nandkar, and Shivam Raikundalia. 2022. Hot-plug events in a pool of reconfigurable data flow resources. Retrieved July 31, 2023 from https://patentimages.storage.googleapis.com/c3/26/ee/3a19bad1548112/US20230205585A1.pdf Patent No. US11487694B1, Filed December 17, 2021, Issued November 1, 2022.

[23] Oliver Peckham. 2022. SambaNova launches Second-Gen DataScale System. *HPCWire* (2022). https://www.hpcwire.com/2022/09/14/sambanova-launches-second-gen-datascale-system/

[24] Martin Russell Raumann, Qi Zheng, Bandish B Shah, Ravinder Kumar, Kin Hing Leung, Sumti Jairath, and Gregory Frederick Grohoski. 2021. Dataflow all-reduce for reconfigurable processor systems. Retrieved July 31, 2023 from https://patentimages.storage.googleapis.com/c3/26/ee/3a19bad1548112/US20230205585A1.pdf Patent No. US11237880B1, Filed July 19, 2021, Issued February 1, 2022.

[25] Hugo Sadok, Zhipeng Zhao, Valerie Choung, Nirav Atre, Daniel S. Berger, James C. Hoe, Aurojit Panda, and Justine Sherry. 2021. We Need Kernel Interposition over the Network Dataplane. In *Proceedings of the Workshop on Hot Topics in Operating Systems* (Ann Arbor, Michigan) *(HotOS '21)*. Association for Computing Machinery, New York, NY, USA, 152–158. https://doi.org/10.1145/3458336.3465281

[26] Arman Shehabi, Sarah J Smith, Eric Masanet, and Jonathan Koomey. 2018. Data center growth in the United States: decoupling the demand for services from electricity use. *Environmental Research Letters* 13, 12 (dec 2018), 124030. https://doi.org/10.1088/1748-9326/aaec9c

[27] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. [n. d.]. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. ([n. d.]).

[28] Jaspal Subhlok, James M Stichnoth, David R O'hallaron, and Thomas Gross. 1993. Exploiting task and data parallelism on a multicomputer. In *Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming*. 13–22.

[29] M S Vinaya, Nagavijayalakshmi Vydyanathan, and Mrugesh Gajjar. 2012. An evaluation of CUDA-enabled virtualization solutions. In *2012 2nd IEEE International Conference on Parallel, Distributed and Grid Computing*. 621–626. https://doi.org/10.1109/PDGC.2012.6449892

[30] Mark Wijtvliet, Henk Corporaal, and Akash Kumar. 2022. CGRA Background and Related Work. *Blocks, Towards Energy-efficient, Coarse-grained Reconfigurable Architectures* (2022), 15–60.

[31] Michael R Wyatt, Valen Yamamoto, Zoë Tosi, Ian Karlin, and Brian Van Essen. 2021. Is disaggregation possible for HPC cognitive simulation?. In *2021 IEEE/ACM Workshop on Machine Learning in High Performance Computing Environments (MLHPC)*. IEEE, 94–105.

[32] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch FSDP: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).

A SCALING STUDY DATA

Tables 1 and 2 are summaries of the strong and weak scaling data we collected.

B SINGLE RDU LATENCY STUDY DATA

Table 3 shows the raw data from the latency study we performed. The study involved running a logistic regression application 100000 times with batch size 1.

RDUs	Per-worker Batch Size	Global Batch Size	E2E Time (sec)	Throughput (samples/sec)	Strong-scaling Speedup	Strong-scaling Efficiency
1	8192	8192	291505.40	2877.68	1.00	100.00%
4	2048	8192	72962.69	11497.12	3.99	99.88%
8	1024	8192	36685.74	22866.13	7.95	99.33%
16	512	8192	18553.87	45212.17	15.71	98.20%
32	256	8192	9417.76	89072.26	30.95	96.73%
64	128	8192	4853.36	172841.15	60.06	93.85%

Table 1: Strong scaling study raw data

RDUs	Per-worker Batch Size	Global Batch Size	E2E Time (sec)	Throughput	Weak-scaling Speedup	Weak-scaling Efficiency
1	128	128	4649.97	2818.77	1	100.00%
4	128	512	4709.86	11131.70	3.95	98.73%
8	128	1024	4733.56	22151.95	7.86	98.23%
16	128	2048	4781.85	43856.46	15.56	97.24%
32	128	4096	4874.28	86049.74	30.53	95.40%
64	128	8192	4881.88	171831.50	60.95	95.25%

Table 2: Weak Scaling study raw data

Category	Time (%)	Time (ns)	Time (sec)
Total	100	77403896570	77.40
Python	43.62	33760896570	33.76
Python to C	5.33	4124525778	4.12
HW setup	8.62	6669039523	6.67
Data transfer	14.26	11036911993	11.04
Data Conversion	0.27	207328077	0.21
Run (HW)	27.91	21605194629	21.61

Table 3: Latency study raw data